NMAM Institute of Technology, Nitte

Department of Computer Science and Engineering

Computer Networks Laboratory

LABORATORY MANUAL

Academic Year 2026–27

Name : _____

USN : _____

Semester / Section : _____

INDEX

Expt. No.	Date	Title of the Experiment	Page No.	Signature
Part A – Network Simulation (NS-2 and AWK)				
A1	14/08/2026	Point-to-point network: packet drops	1	
A2	21/08/2026	TCP and UDP traffic: packet counts and throughput	3	
A3	28/08/2026	Ethernet LAN: throughput versus data rate and error rate	6	
A4	04/09/2026	Ethernet LAN: congestion window of TCP Vegas and Reno	8	
Part B – Network Programming and Routing				
B1	11/09/2026	Bit stuffing and destuffing	10	
B2	09/10/2026	Error detection using CRC (CRC-CCITT)	11	
B3	07/10/2026	Frame sorting technique used in buffers	13	
B4		File transfer using TCP sockets	14	
B5		Calculator using UDP sockets	16	
B6	06/11/2026	Distance vector routing algorithm	19	

Experiment No. A1

Point-to-point network: packet drops

Aim: Simulate a three-node point-to-point network with duplex links between them using NS-2 (Network Simulator 2). Set the queue size, vary the link bandwidth, and determine the number of packets dropped using an AWK script.

Program: exp1.tcl

```
# Experiment 1: Point-to-Point Network & Packet Drop Analysis
# Topology: n0 --- n1 --- n2

set ns [new Simulator]

# Open files for NAM and TRACE logging
set nf [open out.nam w]
$ns namtrace-all $nf

set tf [open out.tr w]
$ns trace-all $tf

# Finish procedure to flush trace and open NAM animator
proc finish {} {
    global ns nf tf
    $ns flush-trace
    close $nf
    close $tf
    exec nam out.nam &
    exit 0
}

# Create 3 nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]

# Create duplex links between nodes (1Mb bandwidth, 10ms delay, DropTail queue)
$ns duplex-link $n0 $n1 1Mb 10ms DropTail
$ns duplex-link $n1 $n2 1Mb 10ms DropTail

# Set orientation for visualization in NAM
$ns duplex-link-op $n0 $n1 orient right-up
$ns duplex-link-op $n1 $n2 orient right-down

# Set queue limit (default is 50 packets)
$ns queue-limit $n0 $n1 50
$ns queue-limit $n1 $n2 50

# Create transport agent: UDP on sender n0, Null on receiver n2
set udp0 [new Agent/UDP]
$ns attach-agent $n0 $udp0

set null0 [new Agent/Null]
$ns attach-agent $n2 $null0
$ns connect $udp0 $null0

# Create Application to generate traffic: CBR (Constant Bit Rate)
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.005
$cbr0 attach-agent $udp0

# Schedule traffic generation events
$ns at 0.5 "$cbr0 start"
$ns at 4.5 "$cbr0 stop"

# Schedule simulation end
$ns at 5.0 "finish"

# Run simulation
$ns run
```

Program: exp1.awk

```
BEGIN {
    count = 0;
}
{
    # Field $1 represents event type ('d' for packet drop)
    # Field $5 is packet type (cbr, tcp, ack, etc.)
    # Field $11 is packet unique identifier
    if ($1 == "d") {
        count++;
        printf("%s\t %s\n", $5, $11);
    }
}
END {
    printf("The number of packets dropped =%d \n", count);
}
```

Output

Case 1: Standard Bandwidth (1 Mbps on both links)

```
$ ns exp1.tcl
$ awk -f exp1.awk out.tr
The number of packets dropped =0
```

Case 2: Bottleneck Bandwidth (0.5 Mbps on link n1-n2)

```
$ ns exp1.tcl
$ awk -f exp1.awk out.tr
cbr         131
cbr         134
cbr         136
cbr         139
...
The number of packets dropped =252
```

Experiment No. A2

TCP and UDP traffic: packet counts and throughput

Aim: Simulate a four-node point-to-point network with links connected as: `n0–n2`, `n1–n2`, and `n2–n3`. Attach a TCP agent between `n0` and `n3`, and a UDP agent between `n1` and `n3`. Generate FTP traffic over TCP and CBR traffic over UDP, analyze packet counts, and calculate network throughput over time using AWK and Xgraph.

Program: exp2.tcl

```
# Experiment 2: TCP and UDP traffic: packet counts and throughput
# Topology:
#   n0 (TCP) \
#             n2 (Router/Queue) --- n3 (Receiver: TCPSink & Null)
#   n1 (UDP) /

set ns [new Simulator]

# Open a new file for NAMTRACE
set nf [open out.nam w]
$ns namtrace-all $nf

# Open a new file to log TRACE
set tf [open out.tr w]
$ns trace-all $tf

# Body of finish procedure
proc finish {} {
    global ns nf tf
    $ns flush-trace
    close $nf
    close $tf
    exec nam out.nam &
    exit 0
}

# Create 4 nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]

# Set node labels
$n0 label "TCP"
$n1 label "UDP"
$n2 label "Receiver"

# Create links between nodes
$ns duplex-link $n0 $n2 1Mb 10ms DropTail
$ns duplex-link $n1 $n2 1Mb 10ms DropTail
$ns duplex-link $n2 $n3 1Mb 10ms DropTail

# Set orientation
$ns duplex-link-op $n0 $n2 orient right-down
$ns duplex-link-op $n1 $n2 orient right-up
$ns duplex-link-op $n2 $n3 orient right

# Set the queue limit - default is 50 packets
$ns queue-limit $n0 $n2 50
$ns queue-limit $n1 $n2 50
$ns queue-limit $n2 $n3 50

# Create TCP agent b/w node 0 and node 3
# Sender
set tcp0 [new Agent/TCP]
$ns attach-agent $n0 $tcp0

# Receiver
set sink0 [new Agent/TCPSink]
$ns attach-agent $n3 $sink0
$ns connect $tcp0 $sink0

# Create FTP Application for TCP Agent
```

```
set ftp0 [new Application/FTP]
$ftp0 attach-agent $tcp0

# Specify TCP packet size
Agent/TCP set packetSize_ 1000

# Create UDP Agent b/w node1 and node3
# Sender
set udp0 [new Agent/UDP]
$ns attach-agent $n1 $udp0

# Receiver
set null0 [new Agent/Null]
$ns attach-agent $n3 $null0
$ns connect $udp0 $null0

# Create CBR Application for UDP Agent
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.005
$cbr0 attach-agent $udp0

# Start and Stop FTP Traffic
$ns at 0.75 "$ftp0 start"
$ns at 4.75 "$ftp0 stop"

# Start and stop CBR traffic
$ns at 0.5 "$cbr0 start"
$ns at 4.5 "$cbr0 stop"

# Stop the simulations
$ns at 5.0 "finish"
$ns run
```

Program: exp2.awk

```
BEGIN {
    tcp_count = 0;
    udp_count = 0;
}
{
    if ($5 == "cbr")
        udp_count++;
    else if ($5 == "tcp")
        tcp_count++;
}
END {
    printf("The number of packets by TCP =%d\n", tcp_count);
    printf("The number of packets by UDP =%d\n", udp_count);
}
```

Program: exp2_throughput.awk

```
BEGIN {
    sSize = 0;
    startTime = 5.0;
    stopTime = 0.1;
    Tput = 0;
}
{
    event = $1;
    time = $2;
    size = $6;
    if (event == "+") {
        if (time < startTime) {
            startTime = time;
        }
    }
    if (event == "r") {
        if (time > stopTime) {
            stopTime = time;
        }
        sSize += size;
        Tput = (sSize / (stopTime - startTime)) * (8 / 1000);
        printf("%f\t %.2f\n", time, Tput);
    }
}
END {
}
```

Output

```
$ ns exp2.tcl
$ awk -f exp2.awk out.tr
The number of packets by TCP =846
The number of packets by UDP =4786

$ awk -f exp2_throughput.awk out.tr > plot_throughput
$ xgraph plot_throughput
$ head -3 plot_throughput; tail -2 plot_throughput
0.500000         -0.00
0.500000         -0.00
0.505000         -0.00
...
4.800920         2017.06
4.809240         2013.24
```

Experiment No. A3

Ethernet LAN: throughput versus data rate and error rate

Aim: Simulate an Ethernet Local Area Network (LAN) using N nodes (6–10 nodes) using IEEE 802.3 CSMA/CD MAC protocol in NS-2. Introduce an error model on the communication link, vary data rates and error rates, and analyze the resulting TCP throughput.

Program: exp3.tcl

```
# Experiment 3: Ethernet LAN: throughput versus data rate and error rate
# Topology: Node n0 connected via duplex link to LAN consisting of nodes n1..n6

set ns [new Simulator]
set namfile [open exp3.nam w]
$ns namtrace-all $namfile
set tracefile [open exp3.tr w]
$ns trace-all $tracefile

proc finish {} {
    global ns namfile tracefile
    $ns flush-trace
    close $namfile
    close $tracefile
    exec nam exp3.nam &
    exec awk -f exp3.awk exp3.tr > exp3.graph &
    exit 0
}

# Create 6 nodes (n0 as sender, n1..n6 as members of Ethernet LAN)
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]
set n5 [$ns node]
set n6 [$ns node]

# Label source and sink nodes
$n0 label "TCPSource"
$n6 label "TCPSink"

# Assign color to flow and nodes
$ns color 1 red
$n0 color blue
$n6 color orange

# Connect n0 to n1 via point-to-point link
$ns duplex-link $n0 $n1 3Mb 20ms DropTail

# Create Ethernet 802.3 CSMA/CD Local Area Network connecting n1 through n6
$ns make-lan "$n1 $n2 $n3 $n4 $n5 $n6" 2Mb 40ms LL Queue/DropTail Mac/802_3

$ns duplex-link-op $n0 $n1 orient right

# Create TCP Agent on n0
set tcp [new Agent/TCP]
$ns attach-agent $n0 $tcp
$tcp set class_ 1

# Create TCPSink Agent on n6
set tcpsink [new Agent/TCPSink]
$ns attach-agent $n6 $tcpsink
$ns connect $tcp $tcpsink

# Attach FTP Application to TCP Agent
set ftp [new Application/FTP]
$ftp attach-agent $tcp

# Error model on the link n0 -> n1 (simulate channel packet losses)
set em [new ErrorModel]
$em unit pkt
$em set rate_ 0.01
```

```
$em ranvar [new RandomVariable/Uniform]
$em drop-target [new Agent/Null]
$ns lossmodel $em $n0 $n1

# Schedule simulation events
$ns at 1.0 "$ftp start"
$ns at 5.0 "$ftp stop"
$ns at 5.5 "finish"
$ns run
```

Program: exp3.awk

```
BEGIN {
    sSize = 0;
    startTime = 5.0;
    stopTime = 0.1;
    Tput = 0;
}
{
    event = $1;
    time = $2;
    size = $6;
    if (event == "+") {
        if (time < startTime) {
            startTime = time;
        }
    }
    if (event == "r") {
        if (time > stopTime) {
            stopTime = time;
        }
        sSize += size;
        Tput = (sSize / (stopTime - startTime)) * (8 / 1000);
        printf("%f\t %.2f\n", time, Tput);
    }
}
END {
}
```

Output

```
$ ns exp3.tcl
$ xgraph exp3.graph
$ head -3 exp3.graph; tail -2 exp3.graph
1.000000         -0.00
1.000000         -0.00
1.020107         15.91
...
5.202373         1328.39
5.206637         1327.12
```

Experiment No. A4

Ethernet LAN: congestion window of TCP Vegas and Reno

Aim: Simulate an Ethernet LAN using N nodes (9 nodes: `n0` to `n8`) with multiple traffic sources. Determine collisions across different nodes, trace the evolution of the congestion window (`cwnd_`) for TCP Vegas and TCP Reno, and compare their delay-based vs loss-based congestion control dynamics using Xgraph.

Program: exp4.tcl

```
# Experiment 4: Ethernet LAN: congestion window of TCP Vegas and Reno
# Topology:
#   n1 (TCP Vegas) \
#                    n0 --- n3 (LAN switch) === [n3, n4, n5, n6, n7 (Sink1), n8 (Sink2)]
#   n2 (TCP Reno)  /

set ns [new Simulator]
set namfile [open exp4.nam w]
$ns namtrace-all $namfile

set tracefile [open exp4.tr w]
$ns trace-all $tracefile

proc finish {} {
    global ns namfile tracefile tfile1 tfile2
    $ns flush-trace
    close $namfile
    close $tracefile
    close $tfile1
    close $tfile2
    exec nam exp4.nam &
    exec awk -f exp4.awk cwnd1.tr > tcp1 &
    exec awk -f exp4.awk cwnd2.tr > tcp2 &
    exit 0
}

# Create the nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]
set n5 [$ns node]
set n6 [$ns node]
set n7 [$ns node]
set n8 [$ns node]

# Color to packet flow
$ns color 1 Blue
$ns color 2 Red

# Set shape and color to node 7 and node 8
$n7 shape Box
$n7 color Blue
$n8 shape hexagon
$n8 color Red

# Create duplex links
$ns duplex-link $n1 $n0 2Mb 10ms DropTail
$ns duplex-link $n2 $n0 2Mb 10ms DropTail
$ns duplex-link $n0 $n3 1Mb 20ms DropTail

# Create LAN
$ns make-lan "$n3 $n4 $n5 $n6 $n7 $n8" 512Kb 40ms LL Queue/DropTail Mac/802_3

# Set Orientation
$ns duplex-link-op $n1 $n0 orient right-down
$ns duplex-link-op $n2 $n0 orient right-up
$ns duplex-link-op $n0 $n3 orient right
$ns queue-limit $n0 $n3 20

# Set transport protocol 1: TCP Vegas
set tcp1 [new Agent/TCP/Vegas]
```

```
$ns attach-agent $n1 $tcp1

set sink1 [new Agent/TCPSink]
$ns attach-agent $n7 $sink1

# Set application 1: FTP over TCP Vegas
set ftp1 [new Application/FTP]
$ftp1 attach-agent $tcp1

$ns connect $tcp1 $sink1
$tcp1 set class_ 1
$tcp1 set packetSize_ 55

set tfile1 [open cwnd1.tr w]
$tcp1 attach $tfile1
$tcp1 trace cwnd_

# Set transport protocol 2: TCP Reno
set tcp2 [new Agent/TCP/Reno]
$ns attach-agent $n2 $tcp2

set sink2 [new Agent/TCPSink]
$ns attach-agent $n8 $sink2

set ftp2 [new Application/FTP]
$ftp2 attach-agent $tcp2

$ns connect $tcp2 $sink2
$tcp2 set class_ 2
$tcp2 set packetSize_ 55

set tfile2 [open cwnd2.tr w]
$tcp2 attach $tfile2
$tcp2 trace cwnd_

# Start and stop traffic flows
$ns at 0.5 "$ftp1 start"
$ns at 1.0 "$ftp2 start"
$ns at 5.0 "$ftp2 stop"
$ns at 5.0 "$ftp1 stop"
$ns at 5.5 "finish"
$ns run
```

Program: exp4.awk

```
BEGIN {
}
{
    # Field $6 contains variable name ("cwnd_")
    # Field $1 contains simulation timestamp
    # Field $7 contains congestion window value
    if ($6 == "cwnd_") {
        printf("%f\t %f\n", $1, $7);
    }
}
END {
}
```

Output

```
$ ns exp4.tcl
$ xgraph tcp1
$ xgraph tcp2
$ head -4 tcp1; tail -2 tcp1
0.000000         1.000000
0.946450         2.000000
0.946450         3.000000
1.392900         4.000000
...
5.483430         1.000000
5.483430         2.000000
```

Experiment No. B1

Bit stuffing and destuffing

Aim: Write a Python program to implement the **Bit Stuffing** (and Destuffing) framing technique used in the Data Link Layer (such as in HDLC / SDLC protocols) to achieve data transparency and prevent unintended frame delimiters (`01111110`).

Program: exp5.py

```
# Experiment 5: Bit Stuffing and Destuffing
# Concept: Insert a '0' after five consecutive '1's at the transmitter,
# and remove that '0' at the receiver to maintain frame delineation.

def bit_stuffing(data):
    stuffed_data = ""
    count = 0
    for bit in data:
        stuffed_data += bit
        if bit == '1':
            count += 1
            if count == 5:
                stuffed_data += '0'
                count = 0
        else:
            count = 0
    return stuffed_data


def bit_destuffing(stuffed_data):
    destuffed_data = ""
    count = 0
    skip_next = False
    for bit in stuffed_data:
        if skip_next:
            skip_next = False
            continue
        destuffed_data += bit
        if bit == '1':
            count += 1
            if count == 5:
                skip_next = True
                count = 0
        else:
            count = 0
    return destuffed_data


if __name__ == "__main__":
    data = input("Enter the input data :")
    print("Original data :", data)
    stuffed = bit_stuffing(data)
    print("Bit Stuffed data :", stuffed)
    destuffed = bit_destuffing(stuffed)
    print("Bit Destuffed data :", destuffed)
```

Output

```
$ python3 exp5.py
Enter the input data :1011111101
Original data : 1011111101
Bit Stuffed data : 10111110101
Bit Destuffed data : 1011111101
```

Experiment No. B2

Error detection using CRC (CRC-CCITT)

Aim: Write a Python program to implement error-detecting code using Cyclic Redundancy Check (CRC). The program computes the CRC remainder using modulo-2 polynomial division, appends it to create the transmitted codeword, simulates channel bit corruption, and verifies the integrity at the receiver side.

Program: exp6.py

```
# Experiment 6: Error Detection using CRC (Cyclic Redundancy Check)
# Concept: Modulo-2 binary polynomial division using XOR operations.
# Divisor generator polynomial: div = [1, 0, 1, 1] (x^3 + x + 1)

div = [1, 0, 1, 1]
cdata = []
n = int(input("Enter the number of bits :"))
data = [int(input(f"Enter bit {i + 1}: ")) for i in range(n)]
m = len(div)

if n < m:
    print("Invalid data ")
else:
    # Append (m - 1) zeros to the data polynomial
    cdata = data.copy()
    for i in range(m - 1):
        cdata.append(0)
    print("Initial Codeword :", cdata)

    # Perform modulo-2 binary division using XOR
    for i in range(n):
        if cdata[i] == 1:
            for j in range(m):
                cdata[i + j] ^= div[j]

    # Append remainder (CRC checksum) to original data
    for i in range(n, n + m - 1):
        data.append(cdata[i])
    print("Codeword to be transmitted is:", data)

    # Optional error injection
    ch = input("Introduce Error - Y -> YES N->NO:")
    if ch.upper() == 'Y':
        p = int(input("Enter the position to change bit :"))
        if 1 <= p <= len(data):
            data[p - 1] ^= 1
        else:
            print("Invalid Position !")

    print("The message received is:", data)

    # Receiver-side verification (modulo-2 division of received codeword)
    for i in range(n):
        if data[i] == 1:
            for j in range(m):
                data[i + j] ^= div[j]

    # Check syndrome remainder
    flag = True
    for i in data:
        if i == 1:
            flag = False
            break

    if flag:
        print("No Error \nSuccessfully Transferred")
    else:
        print("Error !")
```

Output

Run 1: No Error

```
$ python3 exp6.py
Enter the number of bits :4
Enter bit 1: 1
Enter bit 2: 0
Enter bit 3: 0
Enter bit 4: 1
Initial Codeword : [1, 0, 0, 1, 0, 0, 0]
Codeword to be transmitted is: [1, 0, 0, 1, 1, 1, 0]
Introduce Error - Y -> YES N->NO:N
The message received is: [1, 0, 0, 1, 1, 1, 0]
No Error
Successfully Transferred
```

Run 2: Error Introduced

```
$ python3 exp6.py
Enter the number of bits :4
Enter bit 1: 1
Enter bit 2: 0
Enter bit 3: 0
Enter bit 4: 1
Initial Codeword : [1, 0, 0, 1, 0, 0, 0]
Codeword to be transmitted is: [1, 0, 0, 1, 1, 1, 0]
Introduce Error - Y -> YES N->NO:Y
Enter the position to change bit :1
The message received is: [0, 0, 0, 1, 1, 1, 0]
Error !
```

Experiment No. B3

Frame sorting technique used in buffers

Aim: Write a Python program to demonstrate the **Frame Sorting Technique** utilized in receiver buffers of the Data Link and Transport Layers (e.g., TCP reassembly queue). The program partitions a continuous message into fixed-size chunks, assigns sequence numbers, randomly shuffles the frames to simulate out-of-order transmission across an IP network, and sorts the buffer to reconstruct the original stream.

Program: exp7.py

```
# Experiment 7: Frame Sorting Technique Used in Buffers
# Concept: Packets in packet-switched networks traverse diverse paths and may arrive
# out of order. Receiver buffers use sequence numbers to sort and reassemble data.

import random

DATA_SZ = 3


def bubb_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]


if __name__ == "__main__":
    msg = input("Enter the message:")
    # Fragment message into chunks of DATA_SZ characters
    msg_chunks = [msg[i:i + DATA_SZ] for i in range(0, len(msg), DATA_SZ)]

    # Assign sequential frame sequence numbers starting from 1
    frames = list(enumerate(msg_chunks, start=1))
    print("Fragmented frames:", frames)

    # Simulate out-of-order network arrival by shuffling
    for i in range(len(frames)):
        j = random.randint(0, len(frames) - 1)
        frames[i], frames[j] = frames[j], frames[i]
    print("Shuffled frames:", frames)

    # Sort frames in receiver buffer by sequence number
    bubb_sort(frames)
    print("Sorted frames:", frames)

    # Reassemble original stream
    print("Sorted message:" + "".join(x[1] for x in frames))
```

Output

```
$ python3 exp7.py
Enter the message:NMAMIT NITTE
Fragmented frames: [(1, 'NMA'), (2, 'MIT'), (3, ' NI'), (4, 'TTE')]
Shuffled frames: [(2, 'MIT'), (3, ' NI'), (1, 'NMA'), (4, 'TTE')]
Sorted frames: [(1, 'NMA'), (2, 'MIT'), (3, ' NI'), (4, 'TTE')]
Sorted message:NMAMIT NITTE
```

Experiment No. B4

File transfer using TCP sockets

Aim: Using TCP/IP Berkeley sockets in Python, write a client-server program in which the client connects to the server over a reliable stream-oriented TCP connection, requests a file by name, and the server reads the file from disk and streams its contents back to the client. If the requested file is missing, the server sends an error message.

Program: tcp_server.py

```
# tcp_server.py - sends the requested file to the client
import socket
import os

HOST = "127.0.0.1"  # this computer only (localhost)
PORT = 8888  # port number the server listens on
BUFFER_SIZE = 1024  # maximum bytes received by one recv()

# create a TCP socket, bind it to (HOST, PORT) and start listening
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server_socket.bind((HOST, PORT))
server_socket.listen(1)
print("Server is listening on", HOST, "port", PORT)

try:
    while True:
        client_socket, client_address = server_socket.accept()
        with client_socket:
            print("Accepted connection from", client_address)

            # receive the file name sent by the client
            file_name = client_socket.recv(BUFFER_SIZE).decode()
            print("Received file name =", file_name)

            # send the contents if the file exists, otherwise an error text
            if os.path.isfile(file_name):
                with open(file_name, "rb") as file:
                    client_socket.sendall(file.read())
                print("File sent")
            else:
                client_socket.sendall(b"File not found")
                print("File not found")
except KeyboardInterrupt:
    print("\nServer stopped")
finally:
    server_socket.close()
```

Program: tcp_client.py

```
# tcp_client.py - requests a file from the server and saves it
import socket
import sys
import os

SERVER_ADDRESS = ("127.0.0.1", 8888)  # (IP address, port) of the server
BUFFER_SIZE = 1024

# create a TCP socket and connect to the server
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
try:
    client_socket.connect(SERVER_ADDRESS)
except ConnectionRefusedError:
    sys.exit("Could not connect to the server. Is tcp_server.py running?")

file_name = input("Enter the file name: ").strip()
if file_name == "":
    client_socket.close()
    sys.exit("No file name entered.")

# send the file name, then read until the server closes the connection
client_socket.sendall(file_name.encode())
file_contents = b""
```

```
while True:
    chunk = client_socket.recv(BUFFER_SIZE)
    if not chunk:  # empty bytes: the server has closed
        break
    file_contents += chunk
client_socket.close()

if file_contents == b"File not found":
    print("File not found on the server.")
else:
    print("File contents:")
    print(file_contents.decode(errors="replace"))
    save_name = "received_" + os.path.basename(file_name)
    with open(save_name, "wb") as file:
        file.write(file_contents)
    print("File received and saved as '" + save_name + "'")
```

Test file: sample.txt

```
Computer Networks Laboratory
Experiment 8: file transfer using TCP sockets
This file was sent by the server.
```

Output

Terminal 1 (Server)

```
$ python3 tcp_server.py
Server is listening on 127.0.0.1 port 8888
Accepted connection from ('127.0.0.1', 48040)
Received file name = sample.txt
File sent
Accepted connection from ('127.0.0.1', 48042)
Received file name = missing.txt
File not found
^C
Server stopped
```

Terminal 2 (Client)

```
$ python3 tcp_client.py
Enter the file name: sample.txt
File contents:
Computer Networks Laboratory
Experiment 8: file transfer using TCP sockets
This file was sent by the server.
File received and saved as 'received_sample.txt'

$ python3 tcp_client.py
Enter the file name: missing.txt
File not found on the server.
```

Experiment No. B5

Calculator using UDP sockets

Aim: Using UDP sockets in Python, write a client-server calculation service. The client takes two numeric operands and an arithmetic operator (+, −, ⋆, /, %) from standard input, validates inputs locally, and transmits a datagram request to the server. The server processes the arithmetic operation (safely handling division and modulo by zero) and transmits the computed result back to the client over connectionless UDP.

Program: udp_server.py

```
# udp_server.py - receives "number operator number" and replies with the result
import socket

SERVER_IP = "127.0.0.1"
SERVER_PORT = 20001
BUFFER_SIZE = 1024

# create a UDP socket and bind it to (SERVER_IP, SERVER_PORT)
server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server_socket.bind((SERVER_IP, SERVER_PORT))
print("UDP server up and listening on port", SERVER_PORT)

try:
    while True:
        # wait for a datagram; recvfrom() also tells us who sent it
        data, client_address = server_socket.recvfrom(BUFFER_SIZE)
        request = data.decode(errors="replace")
        print("Request from", client_address, ":", request)

        # work out the result (or an error message)
        try:
            num1, operator, num2 = request.split()
            a = float(num1)
            b = float(num2)
            if operator == "+":
                result = a + b
            elif operator == "-":
                result = a - b
            elif operator == "*":
                result = a * b
            elif operator == "/":
                if b == 0:
                    result = "Error: division by zero"
                else:
                    result = a / b
            elif operator == "%":
                if b == 0:
                    result = "Error: modulo by zero"
                else:
                    result = a % b
            else:
                result = "Error: invalid operator"
        except ValueError:
            result = "Error: invalid request"

        print("Result to client:", result)
        server_socket.sendto(str(result).encode(), client_address)
except KeyboardInterrupt:
    print("\nServer stopped")
finally:
    server_socket.close()
```

Program: udp_client.py

```
# udp_client.py - reads two numbers and an operator, asks the server to calculate
import socket
import sys
import math

SERVER_ADDRESS = ("127.0.0.1", 20001)  # (IP address, port) of the server
BUFFER_SIZE = 1024


def is_number(text):
    # True only for finite numbers such as 12, -3.5 or 1e3
    try:
        return math.isfinite(float(text))
    except ValueError:
        return False


# read and check the input before anything is sent
num1 = input("Enter first number : ").strip()
operator = input("Enter operator (+ - * / %) : ").strip()
num2 = input("Enter second number : ").strip()

if not is_number(num1) or not is_number(num2):
    sys.exit("Invalid input: both operands must be numbers.")
if operator not in ("+", "-", "*", "/", "%"):
    sys.exit("Invalid operator: use one of + - * / %")

# create a UDP socket; wait at most 5 seconds for the reply
client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
client_socket.settimeout(5)

request = num1 + " " + operator + " " + num2
client_socket.sendto(request.encode(), SERVER_ADDRESS)

try:
    reply, server_address = client_socket.recvfrom(BUFFER_SIZE)
    print(request, "=", reply.decode())
except (socket.timeout, ConnectionResetError):
    print("No reply from the server. Is udp_server.py running?")
finally:
    client_socket.close()
```

Output

Terminal 1 (Server)

```
$ python3 udp_server.py
UDP server up and listening on port 20001
Request from ('127.0.0.1', 42284) : 12 + 5
Result to client: 17.0
Request from ('127.0.0.1', 42871) : 20 / 4
Result to client: 5.0
Request from ('127.0.0.1', 59562) : 5 / 0
Result to client: Error: division by zero
^C
Server stopped
```

Terminal 2 (Client)

```
$ python3 udp_client.py
Enter first number : 12
Enter operator (+ - * / %) : +
Enter second number : 5
12 + 5 = 17.0

$ python3 udp_client.py
Enter first number : 20
Enter operator (+ - * / %) : /
Enter second number : 4
20 / 4 = 5.0

$ python3 udp_client.py
Enter first number : 5
Enter operator (+ - * / %) : /
Enter second number : 0
5 / 0 = Error: division by zero

$ python3 udp_client.py
Enter first number : abc
Enter operator (+ - * / %) : +
Enter second number : 5
Invalid input: both operands must be numbers.
```

Experiment No. B6

Distance vector routing algorithm

Aim: Write a C program to implement the **Distance Vector Routing Algorithm** (based on the distributed Bellman-Ford algorithm) used in interior gateway protocols like RIP (Routing Information Protocol). The program takes a cost adjacency matrix of a network topology, iteratively updates distances via neighboring nodes, and prints the converged distance table and next-hop for every router.

Program: exp10.c

```c
/*
 * Experiment 10: Distance Vector Routing Algorithm
 * Implementation of distributed Bellman-Ford equation in C:
 *   D_x(y) = min_v { c(x, v) + D_v(y) }
 */

#include <stdio.h>
#include <stdlib.h>

struct node
{
    int dist[20];
    int from[20];
} rt[10];

int main()
{
    int dmat[20][20];
    int n, i, j, k, c = 1;

    printf("Enter the number of nodes: \n");
    if (scanf("%d", &n) != 1) {
        fprintf(stderr, "Invalid input for number of nodes\n");
        return 1;
    }

    printf("Enter the cost matrix: ");
    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= n; j++)
        {
            if (scanf("%d", &dmat[i][j]) != 1) {
                fprintf(stderr, "Invalid input for cost matrix\n");
                return 1;
            }
            dmat[i][i] = 0;
            rt[i].dist[j] = dmat[i][j];
            rt[i].from[j] = j;
        }
    }

    /* Bellman-Ford relaxation loop */
    do
    {
        for (i = 1; i <= n; i++)
        {
            for (j = 1; j <= n; j++)
            {
                for (k = 1; k <= n; k++)
                {
                    if (rt[i].dist[j] > dmat[i][k] + rt[k].dist[j])
                    {
                        rt[i].dist[j] = rt[i].dist[k] + rt[k].dist[j];
                        rt[i].from[j] = k;
                    }
                }
            }
        }
        c++;
    } while (c < n);

    /* Print Distance Table for each router */
```

```
    for (i = 1; i <= n; i++)
    {
        printf("\nDistance Table for router %c is in \n", i + 64);
        for (j = 1; j <= n; j++)
        {
            printf("\tNode %d Via %d. Distance: %d.\n", j, rt[i].from[j], rt[i].dist[j]);
        }
    }

    return 0;
}
```

Output

Run 1: Three Routers (Lab Manual Sample)

```
$ gcc exp10.c -o exp10
$ ./exp10
Enter the number of nodes:
3
Enter the cost matrix: 1 0 1
2 5 3
3 1 0

Distance Table for router A is in
        Node 1 Via 1. Distance: 0.
        Node 2 Via 2. Distance: 0.
        Node 3 Via 3. Distance: 1.

Distance Table for router B is in
        Node 1 Via 1. Distance: 2.
        Node 2 Via 2. Distance: 0.
        Node 3 Via 3. Distance: 3.

Distance Table for router C is in
        Node 1 Via 1. Distance: 3.
        Node 2 Via 2. Distance: 1.
        Node 3 Via 3. Distance: 0.
```

Run 2: Four Routers (No direct link A–C, entered as 999)

```
$ ./exp10
Enter the number of nodes:
4
Enter the cost matrix: 0 2 999 1
2 0 3 7
999 3 0 11
1 7 11 0

Distance Table for router A is in
        Node 1 Via 1. Distance: 0.
        Node 2 Via 2. Distance: 2.
        Node 3 Via 2. Distance: 5.
        Node 4 Via 4. Distance: 1.

Distance Table for router B is in
        Node 1 Via 1. Distance: 2.
        Node 2 Via 2. Distance: 0.
        Node 3 Via 3. Distance: 3.
        Node 4 Via 1. Distance: 3.

Distance Table for router C is in
        Node 1 Via 2. Distance: 5.
        Node 2 Via 2. Distance: 3.
        Node 3 Via 3. Distance: 0.
        Node 4 Via 2. Distance: 6.

Distance Table for router D is in
        Node 1 Via 1. Distance: 1.
        Node 2 Via 1. Distance: 3.
        Node 3 Via 1. Distance: 6.
        Node 4 Via 4. Distance: 0.
```